COMMONS · STRATEGY & ROADMAP

From dashboard to diffusion engine

Strategy document · 6 Oct 2026 · Based on: competitive-landscape.md, the Layer C brief, the Multiplayer AI brief and a new competitor feature teardown · Links: research/strategy-sources.md · † = secondary or vendor-reported

The MVP gives a company two tabs. **Analytics** shows how people use Claude. **Skills** is a library of company skills ranked by runs, with queues for publishing new ones. Since we specced it, the ground has moved under both tabs:

- **Anthropic now ships most of the Skills tab natively.** That includes an org skill inventory sorted by users over 30 days, a publish-request queue that flags duplicates (review became the Enterprise default on 2 Oct 2026), GitHub sync, security scanning, per-skill adoption stats for authors, plugin suggestion tips and an eval tool.
- **All four AI vendors now ship usage analytics with task categories.**

What nobody ships is **the loop that moves practice from the best users to everyone else and proves it stuck**. That's Layer C, the real bottleneck. Shared, forkable runs (multiplayer) are how that loop grows across a company.

> **Commons is the layer that spreads AI practice inside a company. It finds the work your best people already do with Claude and turns it into something colleagues can run. It then puts that in front of the right team when they need it, and proves people are still using it 30 days later.**

What changed since the MVP spec, and what we do about it

MVP CAPABILITY	NATIVE STATUS, OCT 2026	CALL	MOVE
Usage dashboards (Analytics tab)	Anthropic has an admin dashboard, Analytics Chat and the Enterprise Analytics API. OpenAI and Microsoft add task categories, benchmarks and power users†.	THIN	Keep only what feeds targeting. Read the vendor APIs as inputs.
Library ranked by runs	Native Inventory has a "Users (30d)" column and a Most-users sort. Authors see adoption and activity.	STOP	No more library-browsing UI. Show **retention**, which native lacks.
Review queue (`/commons:share-skill`)	Native Requests tab, with a possible-duplicate flag, a reason for each denial, and "Requires review" as the Enterprise default.	STOP	Link out to the native queue. Add our scanner as a PR check.
Suggested queue (repeated work → draft skill)	**Not native.** Larridin finds automation candidates but stops at a recommendation.	DOUBLE DOWN	Drafts built on real examples, eval cases, and private suggestions to the person doing the work
Task classifier	Table stakes (OpenAI task categories, Sep 2026†)	INPUT	Use it to target, not as a product
"Right skill, right moment"	Native tips in Claude Code only. They fire on file and folder signals, are generic, and stop once the plugin is installed.	BUILD	Match on the prompt and show social proof ("people on your team use…")
Proof that it stuck	Native counts users and runs over 30 days only	BUILD	Colleague retention, a holdback group, and "still used by" credit

Five strategic choices

1. **Sell the loop, not the dashboard.** The north-star metric is **retained peer adopters**: colleagues, not the author, who are still running a workflow 30 days after their first run. Usage dashboards are now free from the vendors, and retention is not.
2. **Build on top of Claude's admin console, not beside it.** Read the Analytics API. Write *into* the company's marketplace repo (relevance signals, eval cases, sunset PRs), so that Claude's own surfaces deliver what Commons decides.
3. **Sideways and private by default.** Nothing per-person reaches managers by default. Authors see their own credit, and leaders see aggregates of 5+ people. The research says evaluator visibility suppresses use and that leaderboards get gamed.
4. **Runs before skills.** The thing people share is a **run card** they can fork, because a runnable example beats a taught one and libraries go unused. Multiplayer arrives as watch → comment → fork, not live co-editing.
5. **Claude first, ready for other vendors.** Get the loop working on Claude, where the hooks give us prompt-level signal. The open SKILL.md standard makes it cheap to export to ChatGPT, Gemini and Microsoft 365 later, without building connectors now.

The roadmap at a glance

STAGE	THEME	WHEN	WHAT THE CUSTOMER CAN SAY AT THE END OF IT
0	MVP (built)	now	"We can see how Claude is used and which skills exist."
1	Prove it stuck	weeks 1–6	"We know which skills colleagues kept using, and which big jobs have no skill."
2	Put it in the flow	weeks 7–12	"The right skill reaches the people doing that work inside Claude, and we can show the tips caused the lift."
3	Capture without effort	weeks 13–20	"Our library grows from real work, and it gets better and smaller, not just bigger."
4	Shared, forkable runs	weeks 21–32	"Our best people's work is visible and forkable across teams, not stuck in private chats."
5	Beyond one vendor	after week 32	"One place to spread AI practice across Claude, ChatGPT, Copilot and Gemini."

1 · The market, layer by layer (October 2026)

LAYER	WHO SHIPS IT	STATE	OUR STANCE
A Analytics	Anthropic (admin dashboard, Analytics API with a skills endpoint, Claude Code value tab, leaderboard), OpenAI (power users, benchmarks, task insights†), Microsoft (Copilot Dashboard, Agent 365 "assisted hours"†), Google, Glean, Larridin, Worklytics, and DX and Jellyfish for engineering	Commoditised and free	Treat as input. Never sell "visibility" on its own.
B Internal store	Native org skills at all four vendors. Anthropic's inventory, requests, scanning and eval tool. Notion 3.7 Agent Skills. YC's QM. Manus Project Skills. Tessl. skills.sh	Built into each vendor, walled per vendor	Ride the native store. Add ranking by retention and fork lineage.
C Diffusion	Partly: Section Coach (browser prompt tips), Microsoft's prompt gallery and Viva Engage community (a forum away from the work), Larridin (finds candidates), Claude relevance tips (file signals only)	**Open**	**Own it.** Detect, package, target, reinforce, retain, credit.
M Multiplayer	Claude Tag, ChatGPT Space and group chats†, Teams Mode, Slack Code, Zed Delta, Amp, Linear, Dust, Mosaic, Lore	Crowded in coding and chat channels	Use it to carry Layer C. Run cards and forks, not a live editor.

2 · What Claude's native features don't do (our defensible ground)

1. **Match on the task.** Native tips match files, folders and commands, not what the person is asking. They run only in Claude Code, not in chat or Cowork.
2. **Colleague retention.** There's no cohort view of "tried it, then kept using it", and no split between people trying a skill because peers do and keeping it because it fits.
3. **Capture from repeated work.** Native publishing waits for someone to build a skill.
4. **Social proof in the flow.** "3 people on your team use X for this."
5. **Lift on real work.** Native evals run on hand-written cases. Commons can compare sessions that used a skill with similar ones that didn't.
6. **Credit that can't be gamed.** The native Claude Code leaderboard ranks people by volume.
7. **A view across vendors.** None of the vendors sees the others.

3 · Watch list: who could close the gap

PLAYER	WHY WE WATCH THEM
Anthropic	Ships admin primitives monthly. Claude Reflect (personal fluency) is consumer-only today, and a team version would hit Layer C.
Larridin ($17M, a16z)	Workflow Intelligence finds repeated work and gives Helps/Hurts verdicts. It stops before packaging or delivery.
Notion 3.7 · YC QM	Shared skill library plus admin-gated promotion of skills, exportable to Claude Code
Section	Coaching inside the browser. It can't see Claude Code or Cowork.
Lore · Mosaic (YC S26)	Session → shareable thread → skill. That's closest to our Stage 4.
Glean · Dust	Agent insights with power users by department, plus shared agents
June · Foaster (YC)	Top-down process mapping (June scans company systems; Foaster interviews employees). They don't spread work between peers.

4 · Where to play and how to win

Customer

Companies with **200–5,000 Claude seats** (Team or Enterprise), where most usage is outside engineering. They have a Head of AI or an AI transformation lead, and a renewal or expansion decision within 12 months. Companies that also buy other vendors' seats are the Stage 5 expansion.

Buyer and moment

The Head of AI or CIO sponsors it, with the L&D budget as a second source. Commons is sold **before the seat renewal**, and the pitch is "seats that get used, with proof". Admin decisions stay in Claude's own console, so Commons adds no new governance surface.

Moat over time

A data asset that grows with every customer: **task cluster × skill × who adopted it from whom × still using it at 30 days**. Vendors see only their own runs, and evaluators only see dashboards. With fork lineage and a holdback group, Commons can show which practice caused the lift.

5 · How the customer's value grows: one metric per stage

Coverage

Stage 1: share of active users whose work Commons can see, and share of high-volume task clusters with a skill that colleagues kept using

Lift

Stage 2: first runs among people who got a tip, compared with the 10% holdback group

Retention

Stage 3: share of colleagues still using a skill at 30 days; the library shrinks to what is actually used

Spread

Stage 4: forks per shared run, and days from a power user's run to the fifth colleague running it

The headline number for every stage is **retained peer adopters per 100 seats**. We never report raw runs per person, prompts, or individual rankings.

6 · Roadmap: small features, each 1–3 weeks on top of the MVP

"Feed" = the plugin's existing UserPromptSubmit, PostToolUse and Stop hooks, plus Claude Code telemetry. ✓ = the data is already in the feed. + = needs a new hook, setting or API key.

STAGE 1 Prove it stuck

weeks 1–6 · Analytics tab turns into "what's working"

#	FEATURE	WHAT IT DOES	WHY	DATA
1.1	**Coverage check**	Shows the share of Claude's active users (from the Analytics API) whose sessions reach our hooks and telemetry. Flags machines missing the plugin, and orgs where skill names arrive redacted because `OTEL_LOG_TOOL_DETAILS` is off.	Everything later depends on this. By default, custom skill names arrive as "custom_skill".	+ API key and a managed setting
1.2	**Colleague retention per skill**	Groups first-time users of a skill (excluding the author) by week. A curve shows how many ran it again 22–37 days later. Shown only when 5+ people have used it.	Native features count users, not return use. Fit is what keeps people using a skill.	✓
1.3	**"Still used by" credit, private**	The author sees the number of distinct colleagues (from 2+ groups) still running the skill at 30 days. Raw runs are never shown, and managers see aggregates only.	Leaderboards get gamed, and evaluator visibility suppresses use	✓
1.4	**Coverage map of task clusters**	Ranks task clusters by volume × the share of sessions that used any org skill. "High volume, 0% covered" goes to the top, and each item links to Suggested.	The real backlog is uncovered work, not the library	✓
1.5	**Long-tail card**	"Top 5% of skills = X% of runs. Y% of skills have one user or none." It builds the case for pruning and targeting.	skills.sh: half of skills are installed once	✓
1.6	**Native reconcile**	Pulls daily skill and plugin counts from the API and merges them with hook data, giving runs by surface (chat, Cowork, Code). Library and Review link out to the native console instead of rebuilding it.	Hooks miss chat, and the API misses the task. Stop duplicating native features.	+ Enterprise API (degrades gracefully on Team)

The org sees: which skills colleagues kept using, which large jobs have nothing, and how much of the org Commons can see. **Exit:** retention computed for every skill with 5+ users, and the coverage map reviewed with the Head of AI.

STAGE 2 Put it in the flow

weeks 7–12 · inside Claude, not in a portal

#	FEATURE	WHAT IT DOES	WHY	DATA
2.1	**First-prompt peer tip**	When someone's first prompt matches a task cluster with a retained skill they've never run, the existing UserPromptSubmit hook tells Claude to offer it in one line: "People on your team use /x for this. Want me to run it?" At most once per session and twice per skill per person. Must respond in under 300 ms and do nothing if Commons is down.	Peers drive people to try a skill. Native tips are generic and fire on install only.	✓
2.2	**Repo tip at session start**	A small command hook sends the working directory and repo to Commons, which returns "4 of 6 people who work here use /deploy-check".	Ties a skill to a moment everyone goes through. SessionStart doesn't support http hooks.	+ command hook
2.3	**Relevance auto-writer**	Looks at the commands, folders and files seen in a cluster's sessions and writes Claude's native `relevance` block for that skill, then opens a pull request. Claude's own tips then fire with no Commons code running.	The native engine exists, but nobody knows what signals to give it	✓ + repo write
2.4	**Threshold targeting**	Spends the tip budget on groups at 10–25% adoption. Skips groups under 5% (no peers yet) and over 50% (already spreading).	Adoption takes off in clusters at about 25%	✓ + groups
2.5	**Moment triggers**	Fires on the first session in a new repo, `gh pr create`, or a failed `terraform plan`, and offers the skill tied to that moment.	Libraries spread only when they're tied to moments everyone hits	✓
2.6	**Tip holdback**	A random 10% of eligible people get no tips, so Commons can report causal lift in first runs and in 30-day use.	Turns "tips worked" from an anecdote into evidence	✓

The org sees: skills reaching the people who do that work, with measured lift against a holdback group. **Exit:** lift in first runs from tips is statistically clear in at least one design partner.

STAGE 3 Capture without effort, and keep quality up

weeks 13–20 · the Suggested queue becomes the engine

#	FEATURE	WHAT IT DOES	WHY	DATA
3.1	**Private repeat detector**	After someone does the same kind of task 3 times in 14 days, they alone see "Save this as your skill? /commons:capture". At most once a week, and never reported to admins.	Power users hoard because packaging takes effort. Sharing is sideways, not upward.	✓
3.2	**`/commons:capture`**	Reads the current session transcript and drafts a SKILL.md (steps, tools, inputs, two example prompts) into the person's own skills. One more command shares it.	A runnable workflow beats training. Native creation starts from a blank page.	+ command hook
3.3	**Drafts with real examples**	Each Suggested draft includes the 3 shortest successful real sessions from its cluster, redacted, as examples and test prompts.	Generic LLM drafts are weak	✓
3.4	**Evals from real prompts**	Every draft pull request ships `claude plugin eval` cases built from the cluster's real prompts. CI shows how the results differ with and without the skill.	Native evals need hand-written cases, and we have the real ones	✓ + GitHub Action
3.5	**Friction score and author inbox**	Counts interrupts, failed tools and re-prompts within 2 turns of a skill running. The author gets a private page with redacted examples and a suggested edit as a pull request.	Retention depends on fit, so find bad fit early	✓ + failure hook
3.6	**Sunset queue and pre-PR scanner**	Opens a pull request to deprecate skills with no runs in 45 days or under 10% retention. Each pull request is checked for secrets, PII and risky allowed tools.	A smaller library costs fewer tokens in every session and gets more reuse	✓

The org sees: new skills drawn from real repeated work, tested before they ship, with dead skills retired. **Exit:** at least half of new skills come from captures or suggestions, and the median retention of new skills beats that of older ones.

STAGE 4 Shared, forkable runs (multiplayer)

weeks 21–32 · see the Multiplayer AI brief

#	FEATURE	WHAT IT DOES	WHY	DATA
4.1	**`/commons:share-run`**	Turns a session into a redacted run card: goal, skill, steps, output and outcome. The author previews it first. It has a "Run this yourself" button.	Native sharing is a static snapshot	+ command hook
4.2	**Fork into my skill**	Drafts a personal skill from a card, with a `derived_from` link back. The upstream author gets credit for forkers who are still using it.	The fork is the unit of diffusion and is hard to game	✓ + frontmatter
4.3	**Circles**	Opt-in peer groups of 3–15 people with a feed of run cards and a team threshold bar ("4 of 11 have run this").	Clusters tip over at about 25%. Peers see the work, evaluators don't.	✓
4.4	**Comments and a named reviewer**	Comments on a step, addressed to a person. A shared run that feeds a deliverable needs a named reviewer.	Prevents loafing and workslop	✓
4.5	**Claude Tag → candidates**	Recurring Tag task categories in a channel become Suggested items.	Work done in a channel is already visible to peers	+ Enterprise API
4.6	**Run card vs skill test**	Randomly shows a run card, a SKILL.md or nothing, then measures first runs at 14 days and use at 30.	A first-of-its-kind result, which becomes the sales story	✓

The org sees: the best runs are visible and forkable across teams, with credit flowing to the people who shared. **Exit:** forks per shared run, and days from first run to the fifth colleague, both improving month over month.

STAGE 5 Beyond one vendor

after week 32 · only once Stages 1–4 retain customers

5.1	**Publish once, everywhere**	Exports a published SKILL.md as upload bundles for ChatGPT workspace skills, Gemini Enterprise skills and Microsoft 365 org skills, with an admin checklist for each.	All four vendors shipped org skills in 2026, on the same open standard
5.2	**Coverage import from other vendors**	Imports ChatGPT Enterprise and Copilot usage exports into the same task coverage map.	Most large buyers hold seats from several vendors
5.3	**Watch live**	A read-only live view of a Claude Managed Agents run, where only the owner can take over.	Build only if Stage 4 fork data shows demand

7 · What we stop, defer or refuse

STOP building

- The library-browsing UI and the Review queue (both native now)
- Generic usage charts beyond what targeting needs
- Our own skill hosting. The marketplace repo plus Claude sync already does it.

DEFER

- Connectors to other vendors (Stage 5)
- Live co-steering (Stage 5.3, and maybe never)
- ROI in hours saved. Report holdback lift instead.
- Slack digests (opt-in only, after Stage 2)

REFUSE

- Per-person dashboards for managers
- Leaderboards of runs or prompts
- Showing prompt text to admins beyond 30 days, or without redaction
- Fluency scores attached to named people

8 · Risks and how the plan answers them

RISK	WHY IT'S REAL	ANSWER IN THE PLAN
Anthropic ships Layer C natively	It shipped the inventory, requests, relevance tips and evals in months. A team version of Reflect is plausible.	Build on native features (2.3, 3.4), so our work gets better as Claude's does. Our edge is the cross-team data on who adopted from whom and whether they kept using it. Stay a partner, not a rival.
Hooks and telemetry don't reach everyone	Claude chat has no hooks. Cowork telemetry has no skill event. Skill names are redacted by default.	1.1 coverage check, the 1.6 merge with the API, and honest coverage lines in the UI
Employees feel watched	57% hide their AI use. Works councils (EU) and GDPR apply.	Private by default (1.3, 3.1), groups of 5+ only, redaction, and 30-day deletion of prompts
Tips annoy people	The model may mention them too often, and people tune out.	Hard caps, the 2.6 holdback, and stopping once a skill is retained or dismissed
Team plans lack the Analytics API	The API is Enterprise only	Run on hooks and telemetry alone, and sell the Enterprise extras as an upgrade
Crowded "multiplayer" story	Dust, Notion, QM, Lore and Mosaic all use the language.	Position as "AI practice that spreads", and prove it with the 4.6 experiment.

9 · Bets to test with design partners first

1. Do first-prompt tips (2.1) lift first runs against the holdback group *without* lowering satisfaction with Claude?
2. Will power users share once credit is private and counts colleagues still using their work (1.3) rather than raw runs?
3. Does a run card beat a SKILL.md for people's first run (4.6)?
4. Will a Head of AI pay before renewal for "retained peer adopters per 100 seats"?
5. Is 10–25% adoption the right band to aim tips at in knowledge work, or only in engineering?

10 · Technical facts to check before building

- Whether **http hooks run in Cowork**. Plugin hooks do run there, but support for each hook type isn't documented.
- **SessionEnd hook:** mentioned in the session docs but missing from the hook table.
- **SessionStart** supports only command and MCP-tool hooks, not http, so 2.2 needs a command hook.
- Setting `OTEL_LOG_TOOL_DETAILS=1` through the managed `env` block, so skill names aren't redacted.
- Cowork telemetry has no `skill_activated` event, so we infer runs from `tool_result` or the API.
- The deep-link format for "Run this yourself" sessions with a prefilled prompt.

Key sources (full list: research/strategy-sources.md):

claude.com/docs/plugins/admin, /overview, /share

code.claude.com/docs: plugins/relevance, plugin-evals, plugins/measure, hooks, monitoring-usage, analytics, claude-code-on-the-web

platform.claude.com Enterprise Analytics API

claude.com/docs/cowork/monitoring

OpenAI workspace analytics and Skills help†

Microsoft Agent Dashboard and org skills†

Gemini Enterprise release notes

docs.glean.com agent insights

larridin.com Workflow Intelligence

Section Coach†

tessl.io task evals

vercel.com State of agent skills

TechCrunch on June†

YC Foaster

Layer C brief and Multiplayer AI brief (this folder)